```
// File: frontend-vue/src/App.vue
<script setup lang="ts">
import { RouterLink, RouterView } from 'vue-router'
</script>
<template>
  <div class="app">
    <nav class="navbar">
      <div class="nav-container">
        <RouterLink to="/" class="nav-brand">
            投资估值系统
        </RouterLink>
        <div class="nav-links">
          <RouterLink to="/" class="nav-link">首页</RouterLink>
          <RouterLink to="/valuation" class="nav-link">估值</RouterLink>
          <RouterLink to="/history" class="nav-link">历史记录</RouterLink>
          <RouterLink to="/scenario" class="nav-link">情景分析</RouterLink>
          <RouterLink to="/sensitivity" class="nav-link">敏感性</RouterLink>
          <RouterLink to="/stress-test" class="nav-link">压力测试</RouterLink>
          <RouterLink to="/report" class="nav-link">报告</RouterLink>
        </div>
      </div>
    </nav>
    <main class="main-content">
      <RouterView />
    </main>
    <footer class="footer">
      <p>&copy; 2024 投资估值系统 v1.0 | 股权投资基金专业工具</p>
    </footer>
  </div>
</template>
<style scoped>
.app {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  background: #f5f7fa;
}
.navbar {
  background: white;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
  position: sticky;
  top: 0;
  z-index: 100;
}
.nav-container {
  max-width: 1400px;
  margin: 0 auto;
  padding: 0 20px;
```

```
  display: flex;
  justify-content: space-between;
  align-items: center;
  height: 60px;
}
.nav-brand {
  font-size: 1.2em;
  font-weight: bold;
  color: #667eea;
  text-decoration: none;
  white-space: nowrap;
}
.nav-links {
  display: flex;
  gap: 5px;
}
.nav-link {
  padding: 8px 16px;
  color: #555;
  text-decoration: none;
  border-radius: 6px;
  transition: all 0.3s;
  font-size: 0.9em;
  white-space: nowrap;
}
.nav-link:hover {
  background: #f0f0f0;
  color: #667eea;
}
.nav-link.router-link-active {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
}
.main-content {
  flex: 1;
  padding: 20px 0;
}
.footer {
  background: white;
  padding: 20px;
  text-align: center;
  color: #666;
  font-size: 0.9em;
  border-top: 1px solid #e0e0e0;
}
@media (max-width: 768px) {
  .nav-container {
    flex-direction: column;
```

```
    height: auto;
    padding: 10px 20px;
  }
  .nav-brand {
    margin-bottom: 10px;
  }
  .nav-links {
    flex-wrap: wrap;
    justify-content: center;
  }
}
</style>
<style>
/* Global styles */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
body {
  font-family: 'Microsoft YaHei', 'PingFang SC', 'Helvetica Neue', Arial, sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}
#app {
  min-height: 100vh;
}
</style>
// File: frontend-vue/src/main.ts
import { createApp } from 'vue'
import './style.css'
import App from './App.vue'
import router from './router'
const app = createApp(App)
app.use(router)
app.mount('#app')
// File: frontend-vue/src/router/index.ts
import { createRouter, createWebHistory } from 'vue-router'
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'dashboard',
      component: () => import('../views/Dashboard.vue')
    },
    {
      path: '/valuation',
```

```
      name: 'valuation',
      component: () => import('../views/ValuationInput.vue')
    },
    {
      path: '/valuation/result',
      name: 'valuation-result',
      component: () => import('../views/ValuationResult.vue')
    },
    {
      path: '/history',
      name: 'history',
      component: () => import('../views/History.vue')
    },
    {
      path: '/scenario',
      name: 'scenario',
      component: () => import('../views/ScenarioAnalysis.vue')
    },
    {
      path: '/stress-test',
      name: 'stress-test',
      component: () => import('../views/StressTest.vue')
    },
    {
      path: '/sensitivity',
      name: 'sensitivity',
      component: () => import('../views/SensitivityAnalysis.vue')
    },
    {
      path: '/report',
      name: 'report',
      component: () => import('../views/Report.vue')
    }
  ]
})
export default router
// File: frontend-vue/src/views/Dashboard.vue
<template>
  <div class="dashboard">
    <div class="header">
      <h1>    投资估值系统</h1>
      <p>专业股权投资基金估值工具 v1.0</p>
    </div>
    <div class="cards-grid">
      <div class="card" @click="$router.push('/valuation')">
        <div class="card-icon">   </div>
        <div class="card-title">相对估值</div>
        <div class="card-desc">P/E、P/S、P/B、EV/EBITDA 等方法</div>
```

```
      </div>
      <div class="card" @click="$router.push('/valuation')">
        <div class="card-icon">  </div>
        <div class="card-title">绝对估值</div>
        <div class="card-desc">DCF 现金流折现模型</div>
      </div>
      <div class="card" @click="$router.push('/scenario')">
        <div class="card-icon">  </div>
        <div class="card-title">情景分析</div>
        <div class="card-desc">基准/乐观/悲观情景对比</div>
      </div>
      <div class="card" @click="$router.push('/stress-test')">
        <div class="card-icon">  </div>
        <div class="card-title">压力测试</div>
        <div class="card-desc">收入冲击、毛利率压缩、蒙特卡洛</div>
      </div>
      <div class="card" @click="$router.push('/sensitivity')">
        <div class="card-icon">  </div>
        <div class="card-title">敏感性分析</div>
        <div class="card-desc">单因素/双因素敏感性、龙卷风图</div>
      </div>
      <div class="card" @click="$router.push('/report')">
        <div class="card-icon">  </div>
        <div class="card-title">综合报告</div>
        <div class="card-desc">完整估值分析报告</div>
      </div>
    </div>
    <div class="features">
      <h2>系统功能</h2>
      <ul>
        <li>  多种估值方法交叉验证</li>
        <li>  实时获取 A 股可比公司数据（Tushare）</li>
        <li>  情景分析与压力测试</li>
        <li>  可视化图表展示</li>
        <li>  一键生成综合报告</li>
      </ul>
    </div>
  </div>
</template>
<script setup lang="ts">
</script>
<style scoped>
.dashboard {
  padding: 20px;
  max-width: 1400px;
  margin: 0 auto;
}
.header {
```

```
  text-align: center;
  margin-bottom: 40px;
  padding: 40px 20px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: 12px;
  color: white;
}
.header h1 {
  font-size: 2.5em;
  margin-bottom: 10px;
}
.header p {
  font-size: 1.1em;
  opacity: 0.9;
}
.cards-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 20px;
  margin-bottom: 40px;
}
.card {
  background: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  cursor: pointer;
  transition: all 0.3s;
  text-align: center;
}
.card:hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 24px rgba(102, 126, 234, 0.3);
}
.card-icon {
  font-size: 3em;
  margin-bottom: 15px;
}
.card-title {
  font-size: 1.3em;
  font-weight: bold;
  color: #333;
  margin-bottom: 10px;
}
.card-desc {
  color: #666;
  font-size: 0.9em;
  line-height: 1.5;
```

```
}
.features {
  background: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}
.features h2 {
  color: #333;
  margin-bottom: 20px;
  font-size: 1.5em;
}
.features ul {
  list-style: none;
  padding: 0;
}
.features li {
  padding: 10px 0;
  font-size: 1.1em;
  color: #555;
  border-bottom: 1px solid #f0f0f0;
}
.features li:last-child {
  border-bottom: none;
}
</style>
// File: frontend-vue/src/views/History.vue
<template>
  <div class="history-page">
    <div class="header">
      <h1>    历史记录</h1>
      <p>查看历史估值分析记录</p>
    </div>
    <div class="card">
      <div class="card-actions">
        <button @click="loadHistory" class="btn-refresh" :disabled="loading">
          {{ loading ? '加载中...' : '    刷新' }}
        </button>
      </div>
      <div v-if="history.length === 0 && !loading" class="no-history">
        <p>暂无历史记录</p>
        <p class="hint">进行估值分析后，记录将自动保存在这里</p>
        <router-link to="/valuation" class="btn-primary">开始估值</router-link>
      </div>
      <div v-else-if="history.length > 0" class="history-list">
        <div v-for="item in history" :key="item.id" class="history-item" @click="viewHistoryItem(item.id)">
          <div class="history-index">ID: {{ item.id }}</div>
          <div class="history-header">
```

```
            <span class="history-company">{{ item.company_name }}</span>
            <span class="history-date">{{ formatDate(item.created_at) }}</span>
          </div>
          <div class="history-meta">
            <span class="history-industry">{{ item.industry }}</span>
            <span class="history-stage">{{ item.stage }}</span>
          </div>
          <div class="history-values">
            <span v-if="item.dcf_value" class="value-item">
              DCF: {{ formatMoney(item.dcf_value) }}
            </span>
            <span v-if="item.pe_value" class="value-item">
              P/E: {{ formatMoney(item.pe_value) }}
            </span>
            <span v-if="item.ps_value" class="value-item">
              P/S: {{ formatMoney(item.ps_value) }}
            </span>
          </div>
        </div>
      </div>
    </div>
    <!-- 历史记录详情弹窗 -->
    <div v-if="selectedItem && showModal" class="modal-overlay" @click.self="closeModal">
      <div class="modal-content">
        <div class="modal-header">
          <h2>{{ selectedItem.company_name }} - 估值详情 <span class="modal-index">ID:
{{ selectedItem.id }}</span></h2>
          <button @click="closeModal" class="btn-close">×</button>
        </div>
        <div class="modal-body">
          <div class="detail-grid">
            <div class="detail-item">
              <span class="detail-label">行业</span>
              <span class="detail-value">{{ selectedItem.industry }}</span>
            </div>
            <div class="detail-item">
              <span class="detail-label">阶段</span>
              <span class="detail-value">{{ selectedItem.stage }}</span>
            </div>
            <div class="detail-item">
              <span class="detail-label">营业收入</span>
              <span class="detail-value">{{ formatMoney(selectedItem.revenue) }}</span>
            </div>
            <div class="detail-item">
              <span class="detail-label">创建时间</span>
              <span class="detail-value">{{ formatDateTime(selectedItem.created_at) }}</span>
            </div>
          </div>
```

```
          <div class="valuation-results">
            <h3>估值结果</h3>
            <div v-if="selectedItem.dcf_value" class="result-row">
              <span class="result-label">DCF 估值</span>
              <span class="result-value">{{ formatMoney(selectedItem.dcf_value) }}</span>
            </div>
            <div v-if="selectedItem.pe_value" class="result-row">
              <span class="result-label">P/E 估值</span>
              <span class="result-value">{{ formatMoney(selectedItem.pe_value) }}</span>
            </div>
            <div v-if="selectedItem.ps_value" class="result-row">
              <span class="result-label">P/S 估值</span>
              <span class="result-value">{{ formatMoney(selectedItem.ps_value) }}</span>
            </div>
            <div v-if="selectedItem.pb_value" class="result-row">
              <span class="result-label">P/B 估值</span>
              <span class="result-value">{{ formatMoney(selectedItem.pb_value) }}</span>
            </div>
            <div v-if="selectedItem.ev_value" class="result-row">
              <span class="result-label">EV/EBITDA 估值</span>
              <span class="result-value">{{ formatMoney(selectedItem.ev_value) }}</span>
            </div>
          </div>
        </div>
        <div class="modal-footer">
          <button @click="loadToResultPage" class="btn-primary" :disabled="!hasCompleteData">
            {{ hasCompleteData ? '加载到结果页' : '  此记录无完整详情' }}
          </button>
          <button @click="closeModal" class="btn-secondary">关闭</button>
        </div>
      </div>
    </div>
  </div>
</template>
<script setup lang="ts">
import { ref, onMounted, computed } from 'vue'
import { useRouter } from 'vue-router'
import axios from 'axios'
const router = useRouter()
const history = ref<any[]>([])
const loading = ref(false)
const selectedItem = ref<any>(null)
const showModal = ref(false)
// 计算选中项是否有完整数据
const hasCompleteData = computed(() => {
  return selectedItem.value?.results !== undefined && selectedItem.value?.results !== null
})
onMounted(() => {
```

```
  loadHistory()
})
const loadHistory = async () => {
  loading.value = true
  try {
    const response = await axios.get('http://localhost:8000/api/history?limit=50')
    if (response.data.success) {
      history.value = response.data.history
    }
  } catch (err: any) {
    console.error('加载历史记录失败:', err)
  } finally {
    loading.value = false
  }
}
const viewHistoryItem = async (id: number) => {
  try {
    const response = await axios.get(`http://localhost:8000/api/history/${id}`)
    if (response.data.success) {
      selectedItem.value = response.data.history
      showModal.value = true
    }
  } catch (err: any) {
    console.error('加载历史记录项失败:', err)
  }
}
const closeModal = () => {
  showModal.value = false
  selectedItem.value = null
}
const loadToResultPage = () => {
  if (!selectedItem.value) return
  // 检查是否有完整的 results 数据
  if (!selectedItem.value.results) {
    alert('此历史记录没有完整的估值详情数据。请重新进行估值分析以获取完整数据。')
    return
  }
  // 构建兼容的数据格式：将 results 字段展开到顶层，同时保留 company 信息
  const compatibleData = {
    ...selectedItem.value.results,
    company: {
      name: selectedItem.value.company_name,
      industry: selectedItem.value.industry,
      stage: selectedItem.value.stage,
      revenue: selectedItem.value.revenue
    }
  }
  // 存储到 sessionStorage 并跳转到结果页
```

```
  sessionStorage.setItem('valuationResults', JSON.stringify(compatibleData))
  closeModal()
  router.push('/valuation/result')
}
const formatMoney = (value: number) => {
  if (!value) return '--'
  return (value).toFixed(2) + ' 亿元'
}
const formatDate = (dateStr: string) => {
  if (!dateStr) return '--'
  const date = new Date(dateStr)
  return date.toLocaleDateString('zh-CN', { year: 'numeric', month: '2-digit', day: '2-digit' })
}
const formatDateTime = (dateStr: string) => {
  if (!dateStr) return '--'
  const date = new Date(dateStr)
  return date.toLocaleString('zh-CN', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
    hour: '2-digit',
    minute: '2-digit'
  })
}
</script>
<style scoped>
.history-page {
  padding: 20px;
  max-width: 1200px;
  margin: 0 auto;
}
.header {
  text-align: center;
  margin-bottom: 30px;
  padding: 30px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: 12px;
  color: white;
}
.header h1 {
  font-size: 2em;
  margin-bottom: 10px;
}
.card {
  background: white;
  padding: 25px;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
```

```
}
.card-actions {
  display: flex;
  justify-content: flex-end;
  margin-bottom: 20px;
}
.btn-refresh {
  background: #667eea;
  color: white;
  border: none;
  padding: 8px 16px;
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.3s;
}
.btn-refresh:hover:not(:disabled) {
  background: #5568d3;
}
.btn-refresh:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}
.no-history {
  text-align: center;
  padding: 60px 20px;
  color: #666;
}
.no-history p {
  margin-bottom: 10px;
}
.no-history .hint {
  font-size: 0.9em;
  color: #999;
  margin-bottom: 20px;
}
.btn-primary {
  display: inline-block;
  background: #667eea;
  color: white;
  text-decoration: none;
  padding: 10px 24px;
  border-radius: 6px;
  transition: all 0.3s;
}
.btn-primary:hover {
  background: #5568d3;
  transform: translateY(-2px);
}
```

```
.btn-secondary {
  background: #e0e0e0;
  color: #333;
  border: none;
  padding: 10px 24px;
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.3s;
}
.btn-secondary:hover {
  background: #d0d0d0;
}
.history-list {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(350px, 1fr));
  gap: 20px;
}
.history-item {
  background: #f8f9fa;
  padding: 20px;
  border-radius: 8px;
  cursor: pointer;
  transition: all 0.3s;
  border: 2px solid transparent;
  position: relative;
}
.history-index {
  position: absolute;
  top: 15px;
  right: 15px;
  padding: 4px 10px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  border-radius: 12px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 600;
  font-size: 0.8em;
  box-shadow: 0 2px 8px rgba(102, 126, 234, 0.3);
}
.history-item:hover {
  border-color: #667eea;
  box-shadow: 0 4px 12px rgba(102, 126, 234, 0.2);
  transform: translateY(-2px);
}
.history-header {
  display: flex;
```

```
  justify-content: space-between;
  align-items: center;
  margin-bottom: 10px;
}
.history-company {
  font-size: 1.1em;
  font-weight: 600;
  color: #333;
}
.history-date {
  font-size: 0.85em;
  color: #999;
}
.history-meta {
  display: flex;
  gap: 10px;
  margin-bottom: 12px;
}
.history-industry,
.history-stage {
  font-size: 0.85em;
  padding: 3px 10px;
  background: #e8f0ff;
  color: #667eea;
  border-radius: 12px;
}
.history-values {
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}
.value-item {
  font-size: 0.9em;
  color: #555;
  background: white;
  padding: 6px 12px;
  border-radius: 4px;
}
/* 弹窗样式 */
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0, 0, 0, 0.5);
  display: flex;
  align-items: center;
```

```
  justify-content: center;
  z-index: 1000;
}
.modal-content {
  background: white;
  border-radius: 12px;
  max-width: 600px;
  width: 90%;
  max-height: 80vh;
  overflow-y: auto;
}
.modal-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 25px;
  border-bottom: 1px solid #e0e0e0;
}
.modal-header h2 {
  margin: 0;
  font-size: 1.3em;
  color: #333;
}
.modal-index {
  display: inline-block;
  padding: 2px 8px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  border-radius: 10px;
  font-size: 0.75em;
  margin-left: 8px;
  vertical-align: middle;
}
.btn-close {
  background: none;
  border: none;
  font-size: 1.8em;
  cursor: pointer;
  color: #999;
  width: 32px;
  height: 32px;
  padding: 0;
  line-height: 1;
}
.btn-close:hover {
  color: #333;
}
.modal-body {
```

```
  padding: 25px;
}
.detail-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 15px;
  margin-bottom: 25px;
}
.detail-item {
  display: flex;
  flex-direction: column;
  gap: 5px;
}
.detail-label {
  font-size: 0.85em;
  color: #999;
}
.detail-value {
  font-size: 1em;
  color: #333;
  font-weight: 500;
}
.valuation-results {
  margin-top: 20px;
}
.valuation-results h3 {
  margin-bottom: 15px;
  font-size: 1.1em;
  color: #333;
}
.result-row {
  display: flex;
  justify-content: space-between;
  padding: 12px 15px;
  background: #f8f9fa;
  border-radius: 6px;
  margin-bottom: 8px;
}
.result-label {
  color: #666;
}
.result-value {
  font-weight: 600;
  color: #667eea;
}
.modal-footer {
  display: flex;
  justify-content: flex-end;
```

```
  gap: 10px;
  padding: 15px 25px;
  border-top: 1px solid #e0e0e0;
}
</style>
// File: frontend-vue/src/views/ScenarioAnalysis.vue
<template>
  <div class="scenario-analysis">
    <div class="header">
      <h1>    情景分析</h1>
      <p>基准/乐观/悲观情景估值对比</p>
    </div>
    <!-- 参数配置 -->
    <div class="card config-card">
      <div class="card-title">
          参数配置
        <button @click="showConfig = !showConfig" class="btn-toggle">
          {{ showConfig ? '收起 ▲' : '展开 ▼' }}
        </button>
      </div>
      <div v-show="showConfig" class="config-content">
        <div class="scenario-config-list">
          <div class="scenario-item">
            <div class="scenario-label">乐观情景</div>
            <div class="scenario-inputs">
              <div class="input-group">
                <label>收入增长调整</label>
                <input v-model.number="params.bull.revenue_growth_adj" type="number" step="5" />
                <span class="input-unit">%</span>
              </div>
              <div class="input-group">
                <label>利润率调整</label>
                <input v-model.number="params.bull.margin_adj" type="number" step="1" />
                <span class="input-unit">%</span>
              </div>
              <div class="input-group">
                <label>WACC 调整</label>
                <input v-model.number="params.bull.wacc_adj" type="number" step="0.5" />
                <span class="input-unit">%</span>
              </div>
            </div>
          </div>
          <div class="scenario-item">
            <div class="scenario-label">悲观情景</div>
            <div class="scenario-inputs">
              <div class="input-group">
                <label>收入增长调整</label>
                <input v-model.number="params.bear.revenue_growth_adj" type="number" step="5" />
```

```
              <span class="input-unit">%</span>
            </div>
            <div class="input-group">
              <label>利润率调整</label>
              <input v-model.number="params.bear.margin_adj" type="number" step="1" />
              <span class="input-unit">%</span>
            </div>
            <div class="input-group">
              <label>WACC 调整</label>
              <input v-model.number="params.bear.wacc_adj" type="number" step="0.5" />
              <span class="input-unit">%</span>
            </div>
          </div>
        </div>
      </div>
      <div class="config-hint">
          参数调整说明：正数表示增加，负数表示减少。例如：收入增长调整 +20% 表示在基准增长率基础上增加
20 个百分点
      </div>
      <div class="config-actions">
        <button @click="runAnalysis" class="btn-primary" :disabled="loading">
          {{ loading ? '分析中...' : '重新运行分析' }}
        </button>
      </div>
    </div>
  </div>
  <div class="card">
    <div class="card-title">情景对比</div>
    <div ref="scenarioChart" class="chart"></div>
  </div>
  <div class="card">
    <div class="card-title">情景详情</div>
    <div class="scenario-table-container">
      <table class="scenario-table">
        <thead>
          <tr>
            <th>情景</th>
            <th>估值结果</th>
            <th>收入增长调整</th>
            <th>利润率调整</th>
            <th>WACC 调整</th>
          </tr>
        </thead>
        <tbody>
          <tr v-for="(scenario, name) in scenarios" :key="name" v-show="name !==
'statistics'" :class="getScenarioClass(name)">
            <td class="scenario-name-cell">
              <span class="scenario-badge" :class="getScenarioClass(name)">{{ name }}</span>
```

```
            </td>
            <td class="scenario-value-cell">{{ formatMoney(scenario.valuation?.value || scenario.value) }}</td>
            <td class="scenario-param-cell">
              <span v-if="scenario.scenario && scenario.scenario.revenue_growth_adj !== undefined"
                    :class="getParamClass(scenario.scenario.revenue_growth_adj)">
                {{ formatPercent(scenario.scenario.revenue_growth_adj) }}
              </span>
              <span v-else>--</span>
            </td>
            <td class="scenario-param-cell">
              <span v-if="scenario.scenario && scenario.scenario.margin_adj !== undefined"
                    :class="getParamClass(scenario.scenario.margin_adj)">
                {{ formatPercent(scenario.scenario.margin_adj) }}
              </span>
              <span v-else>--</span>
            </td>
            <td class="scenario-param-cell">
              <span v-if="scenario.scenario && scenario.scenario.wacc_adj !== undefined"
                    :class="getParamClass(scenario.scenario.wacc_adj, true)">
                {{ formatPercent(scenario.scenario.wacc_adj) }}
              </span>
              <span v-else>--</span>
            </td>
          </tr>
        </tbody>
      </table>
    </div>
  </div>
</template>
<script setup lang="ts">
import { ref, onMounted, nextTick } from 'vue'
import * as echarts from 'echarts'
import { scenarioAPI } from '../services/api'
const scenarios = ref<any>({})
const scenarioChart = ref<HTMLElement>()
const chartInstance = ref<echarts.ECharts | null>(null)
const showConfig = ref(false)
const loading = ref(false)
const params = ref({
  bull: {
    revenue_growth_adj: 20,
    margin_adj: 5,
    wacc_adj: -1
  },
  bear: {
    revenue_growth_adj: -20,
    margin_adj: -5,
```

```
    wacc_adj: 2
  }
})
onMounted(async () => {
  const data = sessionStorage.getItem('valuationResults')
  if (data) {
    const parsed = JSON.parse(data)
    // 支持两种数据格式：完整估值结果格式和历史记录格式
    if (parsed.company) {
      // 完整估值结果格式
      if (!parsed.scenario) {
        try {
          // 确保包含所有必填字段（支持历史记录格式）
          const company = parsed.company
          const companyForAPI = {
            name: company.name || '',
            industry: company.industry || '',
            stage: company.stage || '成长期',
            revenue: company.revenue || 0,
            net_income: company.net_income ?? (company.revenue ? company.revenue * 0.15 : 0),
            ebitda: company.ebitda,
            gross_profit: company.gross_profit,
            operating_cash_flow: company.operating_cash_flow,
            total_assets: company.total_assets,
            net_assets: company.net_assets,
            total_debt: company.total_debt ?? 0,
            cash_and_equivalents: company.cash_and_equivalents ?? 0,
            growth_rate: company.growth_rate ?? 0.15,
            margin: company.margin,
            operating_margin: company.operating_margin,
            tax_rate: company.tax_rate ?? 0.25,
            beta: company.beta ?? 1.0,
            risk_free_rate: company.risk_free_rate ?? 0.03,
            market_risk_premium: company.market_risk_premium ?? 0.07,
            cost_of_debt: company.cost_of_debt ?? 0.05,
            target_debt_ratio: company.target_debt_ratio ?? 0.3,
            terminal_growth_rate: company.terminal_growth_rate ?? 0.025
          }
          // 检查是否为多产品模式
          const isMultiProduct = parsed.valuationMode === 'multi' && parsed.products &&
parsed.products.length > 0
          let response
          if (isMultiProduct) {
            // 多产品模式
            console.log('初始化：使用多产品情景分析，产品数量:', parsed.products.length)
            // 转换产品数据：支持百分比整数格式（如 25 -> 0.25）和小数格式
            const productsForAPI = parsed.products.map((p: any) => ({
              ...p,
```

```
            // 转换增长率：如果值 > 1，认为是百分比格式，需要除以 100
            growth_rate_years: (p.growth_rate_years || []).map((g: number) => g > 1 ? g / 100 : g),
            terminal_growth_rate: p.terminal_growth_rate > 1 ? p.terminal_growth_rate / 100 :
p.terminal_growth_rate,
            // 转换利润率
            gross_margin: p.gross_margin > 1 ? p.gross_margin / 100 : p.gross_margin,
            operating_margin: p.operating_margin > 1 ? p.operating_margin / 100 : p.operating_margin,
            // 转换比率
            capex_ratio: p.capex_ratio > 1 ? p.capex_ratio / 100 : p.capex_ratio,
            wc_change_ratio: p.wc_change_ratio > 1 ? p.wc_change_ratio / 100 : p.wc_change_ratio,
            depreciation_ratio: p.depreciation_ratio > 1 ? p.depreciation_ratio / 100 : p.depreciation_ratio,
            // beta 保持原样
            beta: p.beta
          }))
          response = await scenarioAPI.analyze(
            companyForAPI,
            undefined, // 使用默认情景
            productsForAPI,
            companyForAPI.beta,
            0.25,
            companyForAPI.total_debt,
            companyForAPI.cash_and_equivalents
          )
        } else {
          // 单产品模式
          response = await scenarioAPI.analyze(companyForAPI)
        }
        scenarios.value = response.data.results
        sessionStorage.setItem('valuationResults', JSON.stringify({
          ...parsed,
          scenario: response.data
        }))
      } catch (error) {
        console.error('获取情景分析失败:', error)
      }
    } else {
      scenarios.value = parsed.scenario.results
    }
  } else if (parsed.results) {
    // 历史记录格式：直接使用保存的情景数据，不重新调用 API
    scenarios.value = parsed.results
  }
  initChart()
 }
})
const initChart = () => {
 if (!scenarioChart.value) return
 // 销毁旧实例
```

```
  if (chartInstance.value) {
    chartInstance.value.dispose()
  }
  const chart = echarts.init(scenarioChart.value)
  chartInstance.value = chart
  const names: string[] = []
  const values: number[] = []
  for (const [name, data] of Object.entries(scenarios.value)) {
    if (name !== 'statistics') {
      names.push(name)
      const val = (data as any).valuation?.value || (data as any).value || 0
      // 如果值大于 100 万，说明单位是万元，需要转换为亿元
      values.push(val > 1000000 ? val / 10000 : val / 10000)
    }
  }
  chart.setOption({
    title: { text: '情景估值对比', left: 'center' },
    tooltip: { trigger: 'axis', formatter: '{b}: {c} 亿元' },
    xAxis: { type: 'category', data: names },
    yAxis: { type: 'value', name: '估值（亿元）' },
    series: [{
      type: 'bar',
      data: values,
      itemStyle: {
        color: (params: any) => {
          const colors = { '基准': '#5470c6', '乐观': '#91cc75', '悲观': '#ee6666' }
          return colors[params.name as keyof typeof colors] || '#5470c6'
        }
      }
    }]
  })
}
const getScenarioClass = (name: string) => {
  if (name === '乐观') return 'bull'
  if (name === '悲观') return 'bear'
  return 'base'
}
const getParamClass = (value: number, isInvert: boolean = false) => {
  const numValue = typeof value === 'number' ? value : parseFloat(value)
  if (isInvert) {
    // WACC: 负值是好事（降低成本），正值是坏事
    return numValue < 0 ? 'param-positive' : 'param-negative'
  } else {
    // 其他参数：正值是好事，负值是坏事
    return numValue > 0 ? 'param-positive' : 'param-negative'
  }
}
const formatMoney = (value: number) => ((value || 0) / 10000).toFixed(2) + ' 亿元'
```

```
const formatPercent = (value: number | string) => {
  const numValue = typeof value === 'number' ? value : parseFloat(value)
  return (numValue * 100).toFixed(1) + '%'
}
const runAnalysis = async () => {
  loading.value = true
  try {
    const data = sessionStorage.getItem('valuationResults')
    if (!data) {
      alert('请先进行估值分析')
      return
    }
    const parsed = JSON.parse(data)
    let company = parsed.company
    // 确保包含所有必填字段（支持历史记录格式）
    const companyForAPI = {
      name: company.name || '',
      industry: company.industry || '',
      stage: company.stage || '成长期',
      revenue: company.revenue || 0,
      net_income: company.net_income ?? (company.revenue ? company.revenue * 0.15 : 0), // 如果没有，用收
入的 15%估算
      ebitda: company.ebitda,
      gross_profit: company.gross_profit,
      operating_cash_flow: company.operating_cash_flow,
      total_assets: company.total_assets,
      net_assets: company.net_assets,
      total_debt: company.total_debt ?? 0,
      cash_and_equivalents: company.cash_and_equivalents ?? 0,
      growth_rate: company.growth_rate ?? 0.15,
      margin: company.margin,
      operating_margin: company.operating_margin,
      tax_rate: company.tax_rate ?? 0.25,
      beta: company.beta ?? 1.0,
      risk_free_rate: company.risk_free_rate ?? 0.03,
      market_risk_premium: company.market_risk_premium ?? 0.07,
      cost_of_debt: company.cost_of_debt ?? 0.05,
      target_debt_ratio: company.target_debt_ratio ?? 0.3,
      terminal_growth_rate: company.terminal_growth_rate ?? 0.025
    }
    // 构建情景参数数组（基准情景参数固定为 0，乐观和悲观可配置）
    const scenarioParams = [
      {
        name: '基准情景',
        revenue_growth_adj: 0,
        margin_adj: 0,
        wacc_adj: 0
      },
```

```
  {
    name: '乐观情景',
    revenue_growth_adj: params.value.bull.revenue_growth_adj / 100,
    margin_adj: params.value.bull.margin_adj / 100,
    wacc_adj: params.value.bull.wacc_adj / 100
  },
  {
    name: '悲观情景',
    revenue_growth_adj: params.value.bear.revenue_growth_adj / 100,
    margin_adj: params.value.bear.margin_adj / 100,
    wacc_adj: params.value.bear.wacc_adj / 100
  }
]
// 检查是否为多产品模式
const isMultiProduct = parsed.valuationMode === 'multi' && parsed.products && parsed.products.length > 0
let response
if (isMultiProduct) {
  // 多产品模式：传递产品数据
  console.log('使用多产品情景分析，产品数量:', parsed.products.length)
  // 转换产品数据：支持百分比整数格式（如 25 -> 0.25）和小数格式
  const productsForAPI = parsed.products.map((p: any) => ({
    ...p,
    // 转换增长率：如果值 > 1，认为是百分比格式，需要除以 100
    growth_rate_years: (p.growth_rate_years || []).map((g: number) => g > 1 ? g / 100 : g),
    terminal_growth_rate: p.terminal_growth_rate > 1 ? p.terminal_growth_rate / 100 : p.terminal_growth_rate,
    // 转换利润率
    gross_margin: p.gross_margin > 1 ? p.gross_margin / 100 : p.gross_margin,
    operating_margin: p.operating_margin > 1 ? p.operating_margin / 100 : p.operating_margin,
    // 转换比率
    capex_ratio: p.capex_ratio > 1 ? p.capex_ratio / 100 : p.capex_ratio,
    wc_change_ratio: p.wc_change_ratio > 1 ? p.wc_change_ratio / 100 : p.wc_change_ratio,
    depreciation_ratio: p.depreciation_ratio > 1 ? p.depreciation_ratio / 100 : p.depreciation_ratio,
    // beta 保持原样
    beta: p.beta
  }))
  response = await scenarioAPI.analyze(
    companyForAPI,
    scenarioParams,
    productsForAPI,
    companyForAPI.beta,  // company_beta
    0.25,  // tax_rate
    companyForAPI.total_debt,
    companyForAPI.cash_and_equivalents
  )
} else {
  // 单产品模式：原有逻辑
  response = await scenarioAPI.analyze(companyForAPI, scenarioParams)
}
```

```
    scenarios.value = response.data.results
    // 保存到 sessionStorage
    sessionStorage.setItem('valuationResults', JSON.stringify({
      ...parsed,
      scenario: response.data
    }))
    // 重新初始化图表
    await nextTick()
    initChart()
  } catch (error) {
    console.error('情景分析失败:', error)
    alert('情景分析失败，请检查参数设置')
  } finally {
    loading.value = false
  }
}
</script>
<style scoped>
.scenario-analysis {
  padding: 20px;
  max-width: 1200px;
  margin: 0 auto;
}
.header {
  text-align: center;
  margin-bottom: 30px;
  padding: 30px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: 12px;
  color: white;
}
.header h1 {
  font-size: 2em;
  margin-bottom: 10px;
}
.card {
  background: white;
  padding: 25px;
  border-radius: 12px;
  margin-bottom: 20px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}
.config-card {
  /* 高度自适应，宽度保持固定 */
}
.card-title {
  font-size: 1.3em;
  color: #333;
```

```
  margin-bottom: 20px;
  padding-bottom: 10px;
  border-bottom: 2px solid #667eea;
}
.chart {
  height: 400px;
}
/* 情景表格样式 */
.scenario-table-container {
  overflow-x: auto;
}
.scenario-table {
  width: 100%;
  border-collapse: collapse;
  background: white;
}
.scenario-table thead {
  background: #f8f9fa;
}
.scenario-table th {
  padding: 12px 15px;
  text-align: center;
  font-weight: 600;
  color: #555;
  border-bottom: 2px solid #e0e0e0;
}
.scenario-table td {
  padding: 15px;
  text-align: center;
  border-bottom: 1px solid #f0f0f0;
}
.scenario-table tbody tr {
  transition: background 0.2s;
}
.scenario-table tbody tr:hover {
  background: #f8f9fa;
}
.scenario-table tbody tr.bull {
  background: #e8ffe8;
}
.scenario-table tbody tr.bear {
  background: #ffe8e8;
}
.scenario-table tbody tr.base {
  background: #e8f0ff;
}
.scenario-name-cell {
  font-weight: 600;
```

```
}
.scenario-badge {
  display: inline-block;
  padding: 6px 16px;
  border-radius: 20px;
  font-weight: 600;
}
.scenario-badge.base {
  background: #5470c6;
  color: white;
}
.scenario-badge.bull {
  background: #91cc75;
  color: white;
}
.scenario-badge.bear {
  background: #ee6666;
  color: white;
}
.scenario-value-cell {
  font-size: 1.2em;
  font-weight: bold;
  color: #667eea;
}
.scenario-param-cell span {
  display: inline-block;
  padding: 4px 12px;
  border-radius: 4px;
  font-weight: 500;
}
.param-positive {
  color: #27ae60;
  background: #e8f8e8;
}
.param-negative {
  color: #e74c3c;
  background: #fde8e8;
}
.scenario-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}
.scenario-card {
  padding: 20px;
  border-radius: 8px;
  text-align: center;
}
```

```
.scenario-card.base {
  background: #e8f0ff;
  border: 2px solid #5470c6;
}
.scenario-card.bull {
  background: #e8ffe8;
  border: 2px solid #91cc75;
}
.scenario-card.bear {
  background: #ffe8e8;
  border: 2px solid #ee6666;
}
.scenario-header {
  font-size: 1.2em;
  font-weight: bold;
  margin-bottom: 15px;
  color: #333;
}
.scenario-value {
  font-size: 1.8em;
  font-weight: bold;
  color: #667eea;
  margin-bottom: 15px;
}
.scenario-params {
  font-size: 0.9em;
  color: #666;
  line-height: 1.6;
}
/* 参数配置样式 */
.btn-toggle {
  background: transparent;
  color: #667eea;
  border: 1px solid #667eea;
  padding: 4px 12px;
  border-radius: 4px;
  cursor: pointer;
  font-size: 0.85em;
  transition: all 0.3s;
}
.btn-toggle:hover {
  background: #667eea;
  color: white;
}
.config-content {
  margin-top: 20px;
  min-height: 260px;
  max-width: 600px;
```

```
  margin-left: auto;
  margin-right: auto;
  overflow: hidden;
}
.scenario-config-list {
  display: flex;
  flex-direction: row;
  gap: 30px;
  justify-content: center;
}
.scenario-item {
  background: #f8f9fa;
  padding: 12px;
  padding-right: 7px;
  border-radius: 6px;
  border-left: 4px solid #667eea;
  width: 235px;
  flex-shrink: 0;
}
.scenario-label {
  font-weight: 600;
  color: #333;
  margin-bottom: 12px;
  font-size: 0.95em;
}
.scenario-inputs {
  display: flex;
  flex-direction: column;
  gap: 10px;
}
.input-group {
  display: flex;
  align-items: center;
  gap: 8px;
}
.input-group label {
  flex: 0 0 80px;
  font-size: 0.85em;
  color: #555;
  white-space: nowrap;
}
.input-group input {
  flex: 1;
  min-width: 30px;
  max-width: 60px;
  padding: 2px 3px;
  border: 1px solid #ddd;
  border-radius: 3px;
```

```
  font-size: 12px;
  text-align: center;
  box-sizing: border-box;
}
.input-unit {
  flex: 0 0 20px;
  font-size: 0.85em;
  color: #666;
}
.config-hint {
  margin-top: 12px;
  padding: 10px 14px;
  background: #fff3cd;
  border-left: 3px solid #ffc107;
  border-radius: 4px;
  font-size: 0.85em;
  color: #856404;
  line-height: 1.5;
  word-wrap: break-word;
  overflow-wrap: break-word;
  max-width: 500px;
  margin-left: auto;
  margin-right: auto;
}
.config-actions {
  margin-top: 15px;
  text-align: center;
}
.btn-primary {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  border: none;
  padding: 10px 30px;
  border-radius: 6px;
  cursor: pointer;
  font-size: 14px;
  font-weight: 500;
  transition: all 0.3s;
}
.btn-primary:hover:not(:disabled) {
  transform: translateY(-2px);
  box-shadow: 0 5px 15px rgba(102, 126, 234, 0.4);
}
.btn-primary:disabled {
  opacity: 0.6;
  cursor: not-allowed;
}
</style>
```

```
                'data': up_values,
                'itemStyle': {'color': '#91cc75'},
            },
            {
                'name': '负向影响',
                'type': 'bar',
                'data': down_values,
                'itemStyle': {'color': '#ee6666'},
            },
        ],
    }
# ===== 使用示例 =====
if __name__ == "__main__":
    from models import Company, CompanyStage
    # 创建测试公司
    company = Company(
        name="测试股份有限公司",
        industry="软件服务",
        stage=CompanyStage.GROWTH,
        revenue=50000,
        net_income=8000,
        ebitda=12000,
        net_assets=20000,
        growth_rate=0.25,
        operating_margin=0.25,
        beta=1.2,
        terminal_growth_rate=0.025,
    )
    # 创建敏感性分析器
    analyzer = SensitivityAnalyzer(company)
    # 综合敏感性分析
    print("执行综合敏感性分析...")
    results = analyzer.comprehensive_sensitivity_analysis()
    # 显示结果
    print(display_sensitivity_results(results))
    # 双因素敏感性示例
    print("\n【双因素敏感性分析：增长率 vs WACC】")
    two_way = analyzer.two_way_sensitivity('growth_rate', 'wacc', steps=5)
    print(f"  最小估值: {two_way['min_valuation']/10000:.2f}亿元")
    print(f"  最大估值: {two_way['max_valuation']/10000:.2f}亿元")
    print(f"  估值波动范围: {(two_way['max_valuation'] - two_way['min_valuation'])/10000:.2f}亿元")
    # 龙卷风图 JSON 配置（可用于前端 ECharts）
    tornado_json = create_tornado_chart_json(results['tornado_chart'])
    print(f"\n【龙卷风图 ECharts 配置】")
    print(f"  参数数量: {len(tornado_json['yAxis']['data'])}")
    print(f"  系列数量: {len(tornado_json['series'])}")
// File: backend/services/stress_test.py
"""
```

```
压力测试模块
包含收入冲击、毛利率压缩、WACC 冲击等压力测试，以及蒙特卡洛模拟
"""
import numpy as np
from typing import List, Dict, Optional, Any
from core.models import Company, ValuationResult, StressTestResult, MonteCarloResult
from services.absolute_valuation import AbsoluteValuation
class StressTester:
    """压力测试器"""
    def __init__(self, company: Company, valuation_method: str = "DCF"):
        """
        初始化压力测试器
        Args:
            company: 目标公司
            valuation_method: 估值方法
        """
        self.company = company
        self.valuation_method = valuation_method
        # 计算基准估值
        self.base_valuation = self._get_base_valuation()
    def _get_base_valuation(self) -> ValuationResult:
        """获取基准估值"""
        if self.valuation_method == "DCF":
            return AbsoluteValuation.dcf_valuation(self.company)
        else:
            raise ValueError(f"暂不支持{self.valuation_method}方法")
    def revenue_shock_test(
        self,
        shocks: Optional[List[float]] = None
    ) -> List[StressTestResult]:
        """
        收入冲击测试
        Args:
            shocks: 收入变化列表（如-0.3 表示下降 30%）
        Returns:
            压力测试结果列表
        """
        if shocks is None:
            shocks = [-0.3, -0.2, -0.1]
        results = []
        base_value = self.base_valuation.value
        for shock in shocks:
            custom_assumptions = {
                'growth_rate': max(0, self.company.growth_rate * (1 + shock)),
            }
            stressed_valuation = AbsoluteValuation.dcf_valuation(
                self.company, custom_assumptions=custom_assumptions
            )
```

```
        change_pct = (stressed_valuation.value - base_value) / base_value if base_value > 0 else 0
        results.append(StressTestResult(
            test_name="收入冲击测试",
            scenario_description=f"收入{'下降' if shock < 0 else '上升'}{abs(shock):.0%}",
            base_value=base_value,
            stressed_value=stressed_valuation.value,
            change_pct=change_pct,
            details={
                'shock': shock,
                'new_growth_rate': custom_assumptions['growth_rate'],
            }
        ))
    return results
def margin_compression_test(
    self,
    compression_levels: Optional[List[float]] = None
) -> List[StressTestResult]:
    """
    毛利率压缩测试
    Args:
        compression_levels: 利润率压缩幅度列表
    Returns:
        压力测试结果列表
    """
    if compression_levels is None:
        compression_levels = [0.05, 0.10, 0.15]
    results = []
    base_value = self.base_valuation.value
    base_margin = self.company.operating_margin or 0.2
    for compression in compression_levels:
        new_margin = max(0, base_margin - compression)
        custom_assumptions = {
            'operating_margin': new_margin,
        }
        stressed_valuation = AbsoluteValuation.dcf_valuation(
            self.company, custom_assumptions=custom_assumptions
        )
        change_pct = (stressed_valuation.value - base_value) / base_value if base_value > 0 else 0
        results.append(StressTestResult(
            test_name="毛利率压缩测试",
            scenario_description=f"利润率下降{compression:.0%}（从{base_margin:.1%}到{new_margin:.1%}）",
            base_value=base_value,
            stressed_value=stressed_valuation.value,
            change_pct=change_pct,
            details={
                'compression': compression,
                'base_margin': base_margin,
                'new_margin': new_margin,
```

```
                }
            ))
        return results
    def wacc_shock_test(
        self,
        wacc_increases: Optional[List[float]] = None
    ) -> List[StressTestResult]:
        """
        WACC 冲击测试（折现率上升）
        Args:
            wacc_increases: WACC 增加幅度列表
        Returns:
            压力测试结果列表
        """
        if wacc_increases is None:
            wacc_increases = [0.01, 0.02, 0.03]
        results = []
        base_value = self.base_valuation.value
        # 获取基准 WACC
        base_wacc = AbsoluteValuation.calculate_wacc(self.company)
        for increase in wacc_increases:
            new_wacc = base_wacc + increase
            stressed_valuation = AbsoluteValuation.dcf_valuation(
                self.company, wacc=new_wacc
            )
            change_pct = (stressed_valuation.value - base_value) / base_value if base_value > 0 else 0
            results.append(StressTestResult(
                test_name="WACC 冲击测试",
                scenario_description=f"WACC 上升{increase:.1%}（从{base_wacc:.2%}到{new_wacc:.2%}）",
                base_value=base_value,
                stressed_value=stressed_valuation.value,
                change_pct=change_pct,
                details={
                    'wacc_increase': increase,
                    'base_wacc': base_wacc,
                    'new_wacc': new_wacc,
                }
            ))
        return results
    def growth_slowdown_test(
        self,
        slowdown_factors: Optional[List[float]] = None
    ) -> List[StressTestResult]:
        """
        增长放缓测试
        Args:
            slowdown_factors: 增长放缓因子列表（如 0.5 表示增长率减半）
        Returns:
```

```
        压力测试结果列表
    """
    if slowdown_factors is None:
        slowdown_factors = [0.3, 0.5, 0.7]
    results = []
    base_value = self.base_valuation.value
    base_growth = self.company.growth_rate
    for factor in slowdown_factors:
        new_growth = base_growth * factor
        custom_assumptions = {
            'growth_rate': new_growth,
        }
        stressed_valuation = AbsoluteValuation.dcf_valuation(
            self.company, custom_assumptions=custom_assumptions
        )
        change_pct = (stressed_valuation.value - base_value) / base_value if base_value > 0 else 0
        results.append(StressTestResult(
            test_name="增长放缓测试",
            scenario_description=f"增长率降至{factor:.0%}（从{base_growth:.1%}到{new_growth:.1%}）",
            base_value=base_value,
            stressed_value=stressed_valuation.value,
            change_pct=change_pct,
            details={
                'slowdown_factor': factor,
                'base_growth': base_growth,
                'new_growth': new_growth,
            }
        ))
    return results
def extreme_market_crash(self) -> StressTestResult:
    """
    极端市场崩溃情景
    综合考虑收入下降、利润率压缩、WACC 上升等多重负面因素
    Returns:
        压力测试结果
    """
    base_value = self.base_valuation.value
    # 极端情景参数
    revenue_decline = -0.40  # 收入下降 40%
    margin_compression = 0.10  # 利润率压缩 10%
    wacc_increase = 0.03  # WACC 上升 3%
    custom_assumptions = {
        'growth_rate': max(0, self.company.growth_rate * (1 + revenue_decline)),
        'operating_margin': max(0, (self.company.operating_margin or 0.2) - margin_compression),
    }
    base_wacc = AbsoluteValuation.calculate_wacc(self.company)
    stressed_valuation = AbsoluteValuation.dcf_valuation(
        self.company,
```

```
            custom_assumptions=custom_assumptions,
            wacc=base_wacc + wacc_increase
        )
        change_pct = (stressed_valuation.value - base_value) / base_value if base_value > 0 else 0
        return StressTestResult(
            test_name="极端市场崩溃测试",
            scenario_description=f"收入{revenue_decline:.0%}, 利润率-{margin_compression:.0%},
WACC+{wacc_increase:.0%}",
            base_value=base_value,
            stressed_value=stressed_valuation.value,
            change_pct=change_pct,
            details={
                'revenue_decline': revenue_decline,
                'margin_compression': margin_compression,
                'wacc_increase': wacc_increase,
                'new_growth_rate': custom_assumptions['growth_rate'],
                'new_margin': custom_assumptions['operating_margin'],
                'new_wacc': base_wacc + wacc_increase,
            }
        )
    def monte_carlo_simulation(
        self,
        iterations: int = 1000,
        seed: Optional[int] = None
    ) -> MonteCarloResult:
        """
        蒙特卡洛模拟
        对关键参数进行随机采样, 生成估值分布
        Args:
            iterations: 迭代次数
            seed: 随机种子
        Returns:
            蒙特卡洛模拟结果
        """
        if seed:
            np.random.seed(seed)
        values = []
        # 定义参数分布
        growth_rate_mean = self.company.growth_rate
        growth_rate_std = 0.05  # 增长率标准差
        margin_mean = self.company.operating_margin or 0.2
        margin_std = 0.03  # 利润率标准差
        wacc_mean = AbsoluteValuation.calculate_wacc(self.company)
        wacc_std = 0.01  # WACC 标准差
        terminal_growth_mean = self.company.terminal_growth_rate
        terminal_growth_std = 0.005  # 终值增长率标准差
        for _ in range(iterations):
            # 随机采样
```

```
            sample_growth = np.random.normal(growth_rate_mean, growth_rate_std)
            sample_growth = max(0, sample_growth)  # 确保非负
            sample_margin = np.random.normal(margin_mean, margin_std)
            sample_margin = max(0.01, min(0.8, sample_margin))  # 限制在合理范围
            sample_wacc = np.random.normal(wacc_mean, wacc_std)
            sample_wacc = max(0.02, sample_wacc)  # 最小 2%
            sample_terminal = np.random.normal(terminal_growth_mean, terminal_growth_std)
            sample_terminal = max(0, min(0.05, sample_terminal))  # 限制在 0-5%
            # 计算估值
            custom_assumptions = {
                'growth_rate': sample_growth,
                'operating_margin': sample_margin,
            }
            try:
                valuation = AbsoluteValuation.dcf_valuation(
                    self.company,
                    wacc=sample_wacc,
                    terminal_growth_rate=sample_terminal,
                    custom_assumptions=custom_assumptions
                )
                values.append(valuation.value)
            except:
                # 如果计算失败，跳过本次迭代
                continue
        return MonteCarloResult(
            iterations=iterations,
            values=values
        )
    def generate_stress_report(self) -> Dict[str, Any]:
        """
        生成综合压力测试报告
        Returns:
            包含所有压力测试结果的报告
        """
        report = {
            'company': self.company.name,
            'base_valuation': self.base_valuation.value,
            'tests': {}
        }
        # 执行各类测试
        report['tests']['revenue_shock'] = [r.to_dict() for r in self.revenue_shock_test()]
        report['tests']['margin_compression'] = [r.to_dict() for r in self.margin_compression_test()]
        report['tests']['wacc_shock'] = [r.to_dict() for r in self.wacc_shock_test()]
        report['tests']['growth_slowdown'] = [r.to_dict() for r in self.growth_slowdown_test()]
        report['tests']['extreme_crash'] = self.extreme_market_crash().to_dict()
        # 蒙特卡洛模拟
        mc_result = self.monte_carlo_simulation(iterations=1000)
        report['monte_carlo'] = mc_result.to_dict()
```

```
        # 计算最大下行风险
        all_downsides = []
        for test_results in report['tests'].values():
            if isinstance(test_results, list):
                for r in test_results:
                    if r['change_pct'] < 0:
                        all_downsides.append(r['change_pct'])
            elif isinstance(test_results, dict):
                if test_results['change_pct'] < 0:
                    all_downsides.append(test_results['change_pct'])
        if all_downsides:
            report['max_downside'] = min(all_downsides)
        else:
            report['max_downside'] = 0
        return report
# ===== 辅助函数 =====
def display_stress_report(report: Dict[str, Any]) -> str:
    """
    格式化显示压力测试报告
    Args:
        report: 压力测试报告
    Returns:
        格式化的报告文本
    """
    output = []
    output.append("=" * 70)
    output.append(f"{report['company']} - 压力测试报告".center(68))
    output.append("=" * 70)
    base_valuation = report['base_valuation']
    output.append(f"\n 基准估值: {base_valuation/10000:.2f}亿元\n")
    # 各项测试结果
    for test_name, test_results in report['tests'].items():
        output.append(f"【{test_name}】")
        if isinstance(test_results, list):
            for result in test_results:
                output.append(f"  {result['scenario_description']}")
                output.append(f"    估值: {result['stressed_value']/10000:.2f}亿元 "
                              f"({result['change_pct']:+.1%})")
        elif isinstance(test_results, dict):
            output.append(f"  {test_results['scenario_description']}")
            output.append(f"  估值: {test_results['stressed_value']/10000:.2f}亿元 "
                          f"({test_results['change_pct']:+.1%})")
        output.append("")
    # 蒙特卡洛结果
    if 'monte_carlo' in report:
        mc = report['monte_carlo']
        output.append("【蒙特卡洛模拟】")
        output.append(f"  迭代次数: {mc['iterations']}")
```

```
        output.append(f"  均值: {mc['mean']/10000:.2f}亿元")
        output.append(f"  中位数: {mc['median']/10000:.2f}亿元")
        output.append(f"  标准差: {mc['std']/10000:.2f}亿元")
        output.append(f"  范围: {mc['min_value']/10000:.2f} - {mc['max_value']/10000:.2f}亿元")
        output.append(f"  90%置信区间: {mc['confidence_interval_90'][0]/10000:.2f} - "
                      f"{mc['confidence_interval_90'][1]/10000:.2f}亿元")
        output.append("")
    # 最大下行风险
    output.append(f"【最大下行风险】{report['max_downside']:.1%}\n")
    output.append("=" * 70)
    return "\n".join(output)
# ===== 使用示例 =====
if __name__ == "__main__":
    from models import Company, CompanyStage
    # 创建测试公司
    company = Company(
        name="测试股份有限公司",
        industry="软件服务",
        stage=CompanyStage.GROWTH,
        revenue=50000,
        net_income=8000,
        ebitda=12000,
        net_assets=20000,
        total_debt=5000,
        cash_and_equivalents=2000,
        growth_rate=0.25,
        operating_margin=0.25,
        beta=1.2,
        terminal_growth_rate=0.025,
    )
    # 创建压力测试器
    tester = StressTester(company)
    # 生成综合报告
    report = tester.generate_stress_report()
    # 显示报告
    print(display_stress_report(report))
    # 单独运行蒙特卡洛模拟（更多迭代）
    print("\n【蒙特卡洛模拟详情】（5000 次迭代）")
    mc_result = tester.monte_carlo_simulation(iterations=5000, seed=42)
    print(f"  均值: {mc_result.mean/10000:.2f}亿元")
    print(f"  25%分位: {mc_result.percentiles['p25']/10000:.2f}亿元")
    print(f"  75%分位: {mc_result.percentiles['p75']/10000:.2f}亿元")
    print(f"  90%置信区间: {mc_result.confidence_interval_90[0]/10000:.2f} - "
          f"{mc_result.confidence_interval_90[1]/10000:.2f}亿元")
// File: backend/services/valuation_engine.py
"""
统一估值引擎
提供统一的估值入口，整合多种估值方法，自动生成交叉验证和综合报告
```

```
"""
from typing import Optional, List, Dict, Any, Tuple
from datetime import datetime
from core.models import Company, Comparable, CompanyStage, ValuationResult, ScenarioConfig
from services.relative_valuation import RelativeValuation
from services.absolute_valuation import AbsoluteValuation
from utils.other_methods import OtherValuationMethods
from services.scenario_analysis import ScenarioAnalyzer, SCENARIOS
from services.stress_test import StressTester
from services.sensitivity_analysis import SensitivityAnalyzer
from data_fetcher import TushareDataFetcher
class ValuationEngine:
    """
    统一估值引擎
    提供一站式估值服务，整合所有估值方法和风险分析功能
    """
    def __init__(self, tushare_token: Optional[str] = None):
        """
        初始化估值引擎
        Args:
            tushare_token: Tushare API Token，用于获取可比公司数据
        """
        self.tushare_token = tushare_token
        self.fetcher = TushareDataFetcher(tushare_token) if tushare_token else None
    def full_valuation(
        self,
        company: Company,
        comparables: Optional[List[Comparable]] = None,
        methods: Optional[List[str]] = None,
        enable_risk_analysis: bool = True,
        industry_for_comparables: Optional[str] = None
    ) -> Dict[str, Any]:
        """
        完整估值流程
        执行多种估值方法、情景分析、压力测试、敏感性分析，生成综合报告
        Args:
            company: 目标公司
            comparables: 可比公司列表（如不提供且配置了 tushare_token 则自动获取）
            methods: 要使用的估值方法列表
            enable_risk_analysis: 是否执行风险分析
            industry_for_comparables: 行业名称（用于自动获取可比公司）
        Returns:
            综合估值报告
        """
        report = {
            'company': company.name,
            'industry': company.industry,
            'stage': company.stage.value,
            'timestamp': datetime.now().isoformat(),
```

```
    'valuation_methods': {},
    'risk_analysis': {},
    'recommendation': {}
}
# ========== 第一步：相对估值 ==========
if comparables is None and self.fetcher and industry_for_comparables:
    print(f"正在从 Tushare 获取{industry_for_comparables}行业可比公司...")
    comparables = self.fetcher.get_comparable_companies(industry_for_comparables)
if comparables:
    print(f"使用{len(comparables)}家可比公司进行相对估值...")
    relative_results = RelativeValuation.auto_comparable_analysis(
        company, comparables, methods
    )
    report['valuation_methods']['relative'] = {
        name: result.to_dict() for name, result in relative_results.items()
    }
# ========== 第二步：绝对估值（DCF）==========
print("执行 DCF 估值...")
try:
    dcf_result = AbsoluteValuation.dcf_valuation(company)
    report['valuation_methods']['absolute'] = {
        'DCF': dcf_result.to_dict()
    }
except Exception as e:
    print(f"DCF 估值失败: {e}")
# ========== 第三步：风险分析 ==========
if enable_risk_analysis:
    print("执行风险分析...")
    # 情景分析
    try:
        scenario_analyzer = ScenarioAnalyzer(company, dcf_result)
        scenario_results = scenario_analyzer.compare_scenarios()
        report['risk_analysis']['scenario'] = scenario_results
    except Exception as e:
        print(f"情景分析失败: {e}")
    # 压力测试
    try:
        stress_tester = StressTester(company)
        stress_report = stress_tester.generate_stress_report()
        report['risk_analysis']['stress_test'] = stress_report
    except Exception as e:
        print(f"压力测试失败: {e}")
    # 敏感性分析
    try:
        sensitivity_analyzer = SensitivityAnalyzer(company)
        sensitivity_results = sensitivity_analyzer.comprehensive_sensitivity_analysis()
        report['risk_analysis']['sensitivity'] = sensitivity_results
    except Exception as e:
```

```
            print(f"敏感性分析失败: {e}")
    # ========== 第四步：交叉验证与建议 ==========
    all_values = []
    method_details = []
    # 收集所有估值结果
    if 'relative' in report['valuation_methods']:
        for method, result in report['valuation_methods']['relative'].items():
            if result['value'] and result['value'] > 0:
                all_values.append(result['value'])
                method_details.append({
                    'method': f"相对估值-{method}",
                    'value': result['value'],
                    'range': (result.get('value_low'), result.get('value_high'))
                })
    if 'absolute' in report['valuation_methods']:
        for method, result in report['valuation_methods']['absolute'].items():
            if result['value'] and result['value'] > 0:
                all_values.append(result['value'])
                method_details.append({
                    'method': f"绝对估值-{method}",
                    'value': result['value'],
                })
    # 计算推荐估值
    if all_values:
        import numpy as np
        final_value = np.median(all_values)
        value_min = min(all_values) * 0.9
        value_max = max(all_values) * 1.1
        report['recommendation'] = {
            'final_value': final_value,
            'value_range': (value_min, value_max),
            'confidence': self._calculate_confidence(all_values),
            'methods_used': len(all_values),
            'method_details': method_details,
        }
    # ========== 第五步：生成文本报告 ==========
    report['text_report'] = self._generate_text_report(report)
    return report
def quick_valuation(
    self,
    company: Company,
    method: str = 'auto'
) -> ValuationResult:
    """
    快速估值
    根据公司阶段自动选择最合适的估值方法
    Args:
        company: 目标公司
```

```
        method: 指定估值方法（'auto'为自动选择）
    Returns:
        估值结果
    """
    stage = company.stage
    if method == 'auto':
        # 根据公司阶段自动选择方法
        if stage == CompanyStage.EARLY:
            # 早期项目：优先使用 P/S 法或 VC 法
            if company.revenue > 0 and company.net_income <= 0:
                method = 'PS'
            else:
                method = 'VC'
        elif stage == CompanyStage.GROWTH:
            # 成长期：优先使用 P/S 法或 DCF
            method = 'PS' if company.net_income <= 0 else 'DCF'
        else:
            # 成熟期/上市：优先使用 P/E 法或 DCF
            method = 'PE' if company.net_income > 0 else 'DCF'
    # 执行估值
    if method == 'DCF':
        return AbsoluteValuation.dcf_valuation(company)
    elif method == 'PE':
        # 需要可比公司
        raise ValueError("PE 法需要可比公司数据")
    elif method == 'PS':
        raise ValueError("PS 法需要可比公司数据")
    elif method == 'VC':
        return OtherValuationMethods.vc_method_with_future_projection(company)
    else:
        raise ValueError(f"不支持的估值方法: {method}")
def auto_fetch_and_valuate(
    self,
    company: Company,
    industry: str
) -> Dict[str, Any]:
    """
    自动获取可比公司并估值
    Args:
        company: 目标公司
        industry: 所属行业
    Returns:
        估值报告
    """
    if not self.fetcher:
        raise ValueError("需要配置 Tushare Token 才能使用此功能")
    # 获取可比公司
    comparables = self.fetcher.get_comparable_companies(industry)
```

```
    if not comparables:
        raise ValueError(f"未找到{industry}行业的可比公司")
    # 执行完整估值
    return self.full_valuation(
        company,
        comparables=comparables,
        industry_for_comparables=industry
    )
def batch_valuation(
    self,
    companies: List[Company],
    comparables: Optional[List[Comparable]] = None
) -> List[Dict[str, Any]]:
    """
    批量估值
    对多个公司进行估值
    Args:
        companies: 公司列表
        comparables: 共用的可比公司列表
    Returns:
        估值结果列表
    """
    results = []
    for i, company in enumerate(companies):
        print(f"\n 正在估值第{i+1}/{len(companies)}家公司: {company.name}")
        try:
            report = self.full_valuation(company, comparables, enable_risk_analysis=False)
            results.append({
                'company': company.name,
                'success': True,
                'report': report
            })
        except Exception as e:
            results.append({
                'company': company.name,
                'success': False,
                'error': str(e)
            })
    return results
def compare_scenarios(
    self,
    company: Company,
    custom_scenarios: Optional[List[ScenarioConfig]] = None
) -> Dict[str, Any]:
    """
    多情景估值对比
    Args:
        company: 目标公司
```

```
        custom_scenarios: 自定义情景列表
    Returns:
        情景对比结果
    """
    analyzer = ScenarioAnalyzer(company)
    if custom_scenarios:
        results = analyzer.compare_scenarios(scenarios=custom_scenarios)
    else:
        results = analyzer.compare_scenarios()
    return results
def generate_report(
    self,
    report_data: Dict[str, Any],
    format: str = 'text'
) -> str:
    """
    生成格式化报告
    Args:
        report_data: 估值报告数据
        format: 报告格式（text/markdown/html）
    Returns:
        格式化的报告文本
    """
    if format == 'text':
        return self._generate_text_report(report_data)
    elif format == 'markdown':
        return self._generate_markdown_report(report_data)
    elif format == 'html':
        return self._generate_html_report(report_data)
    else:
        raise ValueError(f"不支持的格式: {format}")
# ===== 私有辅助方法 =====
def _calculate_confidence(self, values: List[float]) -> str:
    """计算估值置信度"""
    import numpy as np
    std = np.std(values)
    mean = np.mean(values)
    cv = std / mean if mean > 0 else 1
    if cv < 0.1:
        return "高"
    elif cv < 0.2:
        return "中"
    else:
        return "低"
def _generate_text_report(self, report: Dict[str, Any]) -> str:
    """生成文本格式报告"""
    lines = []
    lines.append("=" * 70)
```

```
        lines.append(f"{report['company']} - 估值报告".center(68))
        lines.append("=" * 70)
        lines.append(f"行业: {report['industry']}")
        lines.append(f"阶段: {report['stage']}")
        lines.append(f"时间: {report['timestamp']}")
        lines.append("")
        # 估值方法结果
        lines.append("-" * 70)
        lines.append("【估值方法】")
        lines.append("-" * 70)
        if 'relative' in report.get('valuation_methods', {}):
            for method, result in report['valuation_methods']['relative'].items():
                lines.append(f"\n{method}: {result['value']/10000:.2f}亿元")
        if 'absolute' in report.get('valuation_methods', {}):
            for method, result in report['valuation_methods']['absolute'].items():
                lines.append(f"\n{method}: {result['value']/10000:.2f}亿元")
        # 推荐估值
        if 'recommendation' in report and report['recommendation']:
            rec = report['recommendation']
            lines.append("\n" + "-" * 70)
            lines.append("【估值建议】")
            lines.append("-" * 70)
            lines.append(f"\n 推荐估值: {rec['final_value']/10000:.2f}亿元")
            lines.append(f"估值区间: {rec['value_range'][0]/10000:.2f} - {rec['value_range'][1]/10000:.2f}亿元")
            lines.append(f"置信度: {rec.get('confidence', 'N/A')}")
        # 风险分析
        if 'risk_analysis' in report:
            lines.append("\n" + "-" * 70)
            lines.append("【风险分析】")
            lines.append("-" * 70)
            if 'stress_test' in report['risk_analysis']:
                stress = report['risk_analysis']['stress_test']
                if 'max_downside' in stress:
                    lines.append(f"\n 最大下行风险: {stress['max_downside']:.1%}")
        lines.append("\n" + "=" * 70)
        return "\n".join(lines)
    def _generate_markdown_report(self, report: Dict[str, Any]) -> str:
        """生成 Markdown 格式报告"""
        lines = []
        lines.append(f"# {report['company']} - 估值报告\n")
        lines.append(f"- **行业**: {report['industry']}")
        lines.append(f"- **阶段**: {report['stage']}")
        lines.append(f"- **时间**: {report['timestamp']}\n")
        lines.append("## 估值方法\n")
        if 'recommendation' in report and report['recommendation']:
            rec = report['recommendation']
            lines.append(f"### 推荐估值\n")
            lines.append(f"- **估值**: {rec['final_value']/10000:.2f}亿元")
```

```
            lines.append(f"- **区间**: {rec['value_range'][0]/10000:.2f} - {rec['value_range'][1]/10000:.2f}亿元")
            lines.append(f"- **置信度**: {rec.get('confidence', 'N/A')}\n")
        if 'risk_analysis' in report:
            lines.append("## 风险分析\n")
            if 'scenario' in report['risk_analysis']:
                scenario = report['risk_analysis']['scenario']
                if 'statistics' in scenario:
                    stats = scenario['statistics']
                    lines.append(f"- **均值**: {stats['mean']/10000:.2f}亿元")
                    lines.append(f"- **标准差**: {stats['std']/10000:.2f}亿元")
        return "\n".join(lines)
    def _generate_html_report(self, report: Dict[str, Any]) -> str:
        """生成 HTML 格式报告"""
        html = f"""
        <html>
        <head>
            <title>{report['company']} - 估值报告</title>
            <style>
                body {{ font-family: Arial, sans-serif; margin: 20px; }}
                .header {{ background: #667eea; color: white; padding: 20px; }}
                .section {{ margin: 20px 0; padding: 15px; border: 1px solid #ddd; }}
                .valuation {{ font-size: 24px; color: #667eea; }}
            </style>
        </head>
        <body>
            <div class="header">
                <h1>{report['company']} - 估值报告</h1>
            </div>
            <div class="section">
                <p>行业: {report['industry']} | 阶段: {report['stage']}</p>
            </div>
            <div class="section">
                <h2>推荐估值</h2>
                <p class="valuation">{report['recommendation']['final_value']/10000:.2f} 亿元</p>
            </div>
        </body>
        </html>
        """
        return html
# ===== 使用示例 =====
if __name__ == "__main__":
    from models import Company, Comparable, CompanyStage
    # 创建测试公司
    company = Company(
        name="测试科技股份有限公司",
        industry="软件服务",
        stage=CompanyStage.GROWTH,
        revenue=50000,
```

```
        net_income=8000,
        ebitda=12000,
        net_assets=20000,
        total_debt=5000,
        cash_and_equivalents=2000,
        growth_rate=0.25,
        operating_margin=0.25,
        beta=1.2,
        terminal_growth_rate=0.025,
    )
    # 创建可比公司
    comparables = [
        Comparable(
            name="可比公司 A",
            revenue=80000,
            net_income=15000,
            net_assets=30000,
            pe_ratio=30.0,
            ps_ratio=6.0,
            pb_ratio=4.0,
        ),
        Comparable(
            name="可比公司 B",
            revenue=60000,
            net_income=10000,
            net_assets=25000,
            pe_ratio=25.0,
            ps_ratio=5.0,
            pb_ratio=3.5,
        ),
    ]
    # 创建估值引擎
    engine = ValuationEngine()
    # 执行完整估值
    print("执行完整估值流程...")
    report = engine.full_valuation(company, comparables)
    # 打印文本报告
    print("\n" + report['text_report'])
// File: frontend-vue/src/services/api.ts
import axios from 'axios'
const API_BASE = 'http://localhost:8000'
// 创建 axios 实例
const apiClient = axios.create({
  baseURL: API_BASE,
  headers: {
    'Content-Type': 'application/json'
  }
})
```

```
// 估值相关 API
export const valuationAPI = {
  // 绝对估值 (DCF)
  dcf: (company: any) => apiClient.post('/api/valuation/absolute', company),
  // 相对估值
  relative: (company: any, comparables: any[], methods?: string[]) =>
    apiClient.post('/api/valuation/relative', {
      company,
      comparables,
      methods
    }),
  // 多产品 DCF 估值
  multiProductDCF: (request: any) => apiClient.post('/api/valuation/multi-product-dcf', request),
  // 多方法交叉验证
  compare: (company: any, comparables?: any[]) =>
    apiClient.post('/api/valuation/compare', {
      company,
      comparables
    })
}
// 情景分析 API
export const scenarioAPI = {
  analyze: (company: any, scenarios?: any[], products?: any[], companyBeta?: number, taxRate?: number,
totalDebt?: number, cash?: number) =>
    apiClient.post('/api/scenario/analyze', {
      company,
      scenarios,
      products,
      company_beta: companyBeta,
      tax_rate: taxRate,
      total_debt: totalDebt,
      cash_and_equivalents: cash
    })
}
// 压力测试 API
export const stressTestAPI = {
  // 收入冲击测试
  revenueShock: (company: any, shocks?: number[]) =>
    apiClient.post('/api/stress-test/revenue', {
      company,
      shocks
    }),
  // 蒙特卡洛模拟
  monteCarlo: (company: any, iterations: number = 1000) =>
    apiClient.post('/api/stress-test/monte-carlo', company, { params: { iterations } }),
  // 综合压力测试
  full: (company: any) =>
    apiClient.post('/api/stress-test/full', company)
```

```
}
// 敏感性分析 API
export const sensitivityAPI = {
  // 单因素敏感性分析
  oneWay: (company: any, paramName: string, paramRange?: number[]) =>
    apiClient.post('/api/sensitivity/one-way', {
      company,
      param_name: paramName,
      param_range: paramRange
    }),
  // 龙卷风图数据
  tornado: (company: any, paramChanges?: Record<string, number>) =>
    apiClient.post('/api/sensitivity/tornado', {
      company,
      param_changes: paramChanges
    }),
  // 综合敏感性分析
  comprehensive: (company: any) =>
    apiClient.post('/api/sensitivity/comprehensive', company)
}
// 数据获取 API
export const dataAPI = {
  // 配置 Tushare Token
  configureToken: (token: string) =>
    apiClient.post('/api/data/tushare/configure', null, {
      params: { token }
    }),
  // 获取可比公司
  getComparables: (industry: string, marketCapMin?: number, marketCapMax?: number, limit: number = 20) =>
    apiClient.get(`/api/data/comparable/${industry}`, {
      params: { market_cap_min: marketCapMin, market_cap_max: marketCapMax, limit }
    }),
  // 获取单个股票财务数据（上市公司从 Tushare 导入）
  getStockData: (tsCode: string) =>
    apiClient.get(`/api/data/stock/${tsCode}`),
  // 获取行业估值倍数
  getIndustryMultiples: (industry: string, method: string = 'median') =>
    apiClient.get(`/api/data/industry-multiples/${industry}`, {
      params: { method }
    }),
  // 搜索公司
  search: (keywords: string[], limit: number = 10) =>
    apiClient.post('/api/data/search', { keywords, limit })
}
// 历史记录 API
export const historyAPI = {
  // 获取历史记录列表
  getList: (limit: number = 50) =>
```

```
    apiClient.get(`/api/history`, { params: { limit } }),
  // 获取单个历史记录详情
  getDetail: (historyId: number) =>
    apiClient.get(`/api/history/${historyId}`),
  // 保存完整估值结果到历史记录
  save: (data: any) =>
    apiClient.post('/api/history/save', data)
}
export default apiClient
// File: backend/utils/other_methods.py
"""
其他估值方法模块
包含风险投资法（VC 法）、成本法/净资产法、交易对价参考法等
"""
from typing import Optional, Dict, Any, List
from core.models import Company, ValuationResult
class OtherValuationMethods:
    """其他估值方法类"""
    @staticmethod
    def vc_method(
        company: Company,
        exit_valuation: float,
        target_return_multiple: float = 10.0,
        investment_years: int = 5,
        exit_method: str = "PE",
        exit_multiple: Optional[float] = None
    ) -> ValuationResult:
        """
        风险投资法估值（倒推法）
        从预期退出时的估值倒推当前投后估值
        Args:
            company: 目标公司
            exit_valuation: 预期退出时的估值（万元）
            target_return_multiple: 目标回报倍数
            investment_years: 投资年限
            exit_method: 退出估值方法
            exit_multiple: 退出倍数（如提供则用于计算退出估值）
        Returns:
            估值结果
        """
        # 如果提供退出倍数，重新计算退出估值
        if exit_multiple:
            if exit_method == "PE" and company.net_income > 0:
                # 假设退出时的净利润按当前增长率增长
                future_net_income = company.net_income * ((1 + company.growth_rate) ** investment_years)
                exit_valuation = future_net_income * exit_multiple
            elif exit_method == "PS" and company.revenue > 0:
                future_revenue = company.revenue * ((1 + company.growth_rate) ** investment_years)
```

```
            exit_valuation = future_revenue * exit_multiple
        # 倒推当前估值
        current_valuation = exit_valuation / target_return_multiple
        return ValuationResult(
            method="VC 法",
            value=current_valuation,
            details={
                'exit_valuation': exit_valuation,
                'target_return_multiple': target_return_multiple,
                'investment_years': investment_years,
                'exit_method': exit_method,
                'exit_multiple': exit_multiple,
                'implied_irr': (target_return_multiple ** (1 / investment_years)) - 1,
            },
            assumptions={
                'growth_rate': company.growth_rate,
            }
        )
    @staticmethod
    def vc_method_with_future_projection(
        company: Company,
        projection_years: int = 5,
        target_pe: float = 20.0,
        target_return_multiple: float = 10.0,
        margin_improvement: float = 0.0
    ) -> ValuationResult:
        """
        带预测的风险投资法
        基于未来盈利预测和目标退出倍数进行估值
        Args:
            company: 目标公司
            projection_years: 预测年数
            target_pe: 目标退出时 P/E 倍数
            target_return_multiple: 目标回报倍数
            margin_improvement: 利润率年提升幅度
        Returns:
            估值结果
        """
        # 预测退出时净利润
        future_net_income = company.net_income
        for year in range(projection_years):
            # 收入增长
            future_net_income *= (1 + company.growth_rate)
            # 利润率提升
            if margin_improvement > 0:
                future_net_income *= (1 + margin_improvement)
        # 计算退出估值
        exit_valuation = future_net_income * target_pe
```

```
    # 倒推当前估值
    current_valuation = exit_valuation / target_return_multiple
    return ValuationResult(
        method="VC 法（预测）",
        value=current_valuation,
        details={
            'future_net_income': future_net_income,
            'target_pe': target_pe,
            'exit_valuation': exit_valuation,
            'target_return_multiple': target_return_multiple,
            'projection_years': projection_years,
        },
        assumptions={
            'growth_rate': company.growth_rate,
            'margin_improvement': margin_improvement,
        }
    )
@staticmethod
def cost_method(
    company: Company,
    intangible_asset_value: float = 0,
    goodwill_value: float = 0,
    adjustment_factor: float = 1.0
) -> ValuationResult:
    """
    成本法/净资产法估值
    基于公司净资产价值进行估值，适用于资产密集型企业
    Args:
        company: 目标公司
        intangible_asset_value: 无形资产评估价值
        goodwill_value: 商誉价值
        adjustment_factor: 调整系数
    Returns:
        估值结果
    """
    if not company.net_assets:
        raise ValueError("缺少净资产数据")
    # 基础净资产价值
    base_value = company.net_assets
    # 加上无形资产和商誉
    adjusted_net_assets = base_value + intangible_asset_value + goodwill_value
    # 应用调整系数
    final_value = adjusted_net_assets * adjustment_factor
    return ValuationResult(
        method="成本法/净资产法",
        value=final_value,
        details={
            'net_assets': company.net_assets,
```

```
            'intangible_asset_value': intangible_asset_value,
            'goodwill_value': goodwill_value,
            'adjusted_net_assets': adjusted_net_assets,
            'adjustment_factor': adjustment_factor,
            'price_to_book_ratio': final_value / company.net_assets if company.net_assets > 0 else None,
        },
        assumptions={}
    )
@staticmethod
def adjusted_net_asset_method(
    company: Company,
    asset_adjustments: Dict[str, float],
    liability_adjustments: Dict[str, float]
) -> ValuationResult:
    """
    调整净资产法
    对各项资产和负债进行公允价值调整
    Args:
        company: 目标公司
        asset_adjustments: 资产调整字典 {资产名称: 调整金额}
        liability_adjustments: 负债调整字典
    Returns:
        估值结果
    """
    # 计算调整后的净资产
    total_asset_adjustment = sum(asset_adjustments.values())
    total_liability_adjustment = sum(liability_adjustments.values())
    if company.net_assets:
        adjusted_net_assets = company.net_assets + total_asset_adjustment - total_liability_adjustment
    else:
        raise ValueError("缺少净资产数据")
    return ValuationResult(
        method="调整净资产法",
        value=adjusted_net_assets,
        details={
            'original_net_assets': company.net_assets,
            'asset_adjustments': asset_adjustments,
            'liability_adjustments': liability_adjustments,
            'total_asset_adjustment': total_asset_adjustment,
            'total_liability_adjustment': total_liability_adjustment,
            'adjusted_net_assets': adjusted_net_assets,
        },
        assumptions={}
    )
@staticmethod
def transaction_comparable(
    company: Company,
    transactions: List[Dict[str, Any]]
```

```
) -> ValuationResult:
    """
    交易对价参考法
    参考近期类似交易的估值倍数
    Args:
        company: 目标公司
        transactions: 交易列表，每个交易包含：
            - company_name: 交易公司名称
            - deal_value: 交易金额
            - metric_value: 财务指标值（如收入、利润）
            - multiple: 交易倍数
            - deal_date: 交易日期
            - stage: 交易阶段
    Returns:
        估值结果
    """
    if not transactions:
        raise ValueError("缺少交易数据")
    # 提取交易倍数
    multiples = [t['multiple'] for t in transactions if t.get('multiple')]
    if not multiples:
        raise ValueError("交易数据中缺少倍数信息")
    import numpy as np
    # 计算平均/中位数倍数
    avg_multiple = np.mean(multiples)
    median_multiple = np.median(multiples)
    # 应用到目标公司
    # 优先使用净利润，其次收入
    if company.net_income > 0:
        metric_value = company.net_income
        metric_name = "净利润"
    elif company.revenue > 0:
        metric_value = company.revenue
        metric_name = "营业收入"
    else:
        raise ValueError("缺少可用于估值的财务指标")
    # 使用中位数倍数估值
    valuation = metric_value * median_multiple
    return ValuationResult(
        method="交易对价参考法",
        value=valuation,
        details={
            'transaction_count': len(transactions),
            'avg_multiple': avg_multiple,
            'median_multiple': median_multiple,
            'multiple_range': (min(multiples), max(multiples)),
            'metric_used': metric_name,
            'metric_value': metric_value,
```

```
            'transactions': transactions,
        },
        assumptions={}
    )
@staticmethod
def first_chicago_method(
    company: Company,
    success_scenario: Dict[str, Any],
    failure_scenario: Dict[str, Any],
    probability_of_success: float = 0.3
) -> ValuationResult:
    """
    第一芝加哥法
    针对早期项目，考虑成功和失败两种情景的加权估值
    Args:
        company: 目标公司
        success_scenario: 成功情景估值
        failure_scenario: 失败情景估值
        probability_of_success: 成功概率
    Returns:
        估值结果
    """
    success_value = success_scenario.get('value', 0)
    failure_value = failure_scenario.get('value', 0)
    # 期望价值
    expected_value = (success_value * probability_of_success +
                      failure_value * (1 - probability_of_success))
    return ValuationResult(
        method="第一芝加哥法",
        value=expected_value,
        details={
            'success_value': success_value,
            'failure_value': failure_value,
            'probability_of_success': probability_of_success,
            'success_scenario': success_scenario,
            'failure_scenario': failure_scenario,
        },
        assumptions={}
    )
@staticmethod
def sum_of_parts_valuation(
    company: Company,
    business_units: List[Dict[str, Any]]
) -> ValuationResult:
    """
    分部加总法
    将公司各业务部门分别估值后加总
    Args:
        company: 目标公司
```

```
        business_units: 业务单元列表，每个包含：
            - name: 业务名称
            - revenue: 收入
            - multiple: 估值倍数
            - value: 估值（如有直接估值）
        Returns:
            估值结果
        """
        parts_value = 0
        parts_details = []
        for unit in business_units:
            if unit.get('value'):
                unit_value = unit['value']
            elif unit.get('revenue') and unit.get('multiple'):
                unit_value = unit['revenue'] * unit['multiple']
            else:
                continue
            parts_value += unit_value
            parts_details.append({
                'name': unit['name'],
                'value': unit_value,
                'revenue': unit.get('revenue'),
                'multiple': unit.get('multiple'),
            })
        # 减去公司层面成本（协同效应折扣）
        corporate_discount = parts_value * 0.1  # 假设10%的公司层面成本
        final_value = parts_value - corporate_discount
        return ValuationResult(
            method="分部加总法",
            value=final_value,
            details={
                'parts_value': parts_value,
                'corporate_discount': corporate_discount,
                'business_units': parts_details,
            },
            assumptions={}
        )
# ===== 辅助函数 =====
def analyze_stage_appropriate_valuation(
    company: Company
) -> str:
    """
    根据公司阶段推荐估值方法
    Args:
        company: 目标公司
    Returns:
        推荐的估值方法
    """
```

```
    stage = company.stage
    if stage.value == "早期":
        return "VC 法、交易对价法"
    elif stage.value == "成长期":
        return "P/S 法、DCF 法、VC 法"
    elif stage.value == "成熟期":
        return "P/E 法、DCF 法、EV/EBITDA 法"
    else:  # 上市公司
        return "P/E 法、P/B 法、EV/EBITDA 法、DCF 法"
# ===== 使用示例 =====
if __name__ == "__main__":
    from models import Company, CompanyStage
    print("=" * 60)
    print("其他估值方法示例")
    print("=" * 60)
    # 早期项目
    early_company = Company(
        name="早期科技公司",
        industry="人工智能",
        stage=CompanyStage.EARLY,
        revenue=2000,  # 2000 万
        net_income=-500,  # 亏损
        net_assets=3000,
        growth_rate=0.50,
    )
    print(f"\n【公司】{early_company.name}")
    print(f"推荐估值方法: {analyze_stage_appropriate_valuation(early_company)}")
    # VC 法估值
    vc_result = OtherValuationMethods.vc_method_with_future_projection(
        early_company,
        projection_years=5,
        target_pe=25.0,
        target_return_multiple=15.0,
    )
    print(f"\n{vc_result}")
    print(f"  退出估值: {vc_result.details['exit_valuation']/10000:.2f}亿元")
    print(f"  当前估值: {vc_result.value/10000:.2f}亿元")
    # 成长期项目
    growth_company = Company(
        name="成长期公司",
        industry="软件服务",
        stage=CompanyStage.GROWTH,
        revenue=20000,  # 2 亿
        net_income=3000,  # 3000 万
        net_assets=10000,
        growth_rate=0.40,
    )
    # 交易对价法
```

```
    transactions = [
        {'company_name': 'A 公司', 'deal_value': 120000, 'metric_value': 6000, 'multiple': 20, 'stage': 'B 轮'},
        {'company_name': 'B 公司', 'deal_value': 80000, 'metric_value': 4000, 'multiple': 20, 'stage': 'B 轮'},
        {'company_name': 'C 公司', 'deal_value': 150000, 'metric_value': 5000, 'multiple': 30, 'stage': 'C 轮'},
    ]
    trans_result = OtherValuationMethods.transaction_comparable(growth_company, transactions)
    print(f"\n{trans_result}")
    print(f"  参考交易: {trans_result.details['transaction_count']}笔")
    print(f"  中位数倍数: {trans_result.details['median_multiple']:.1f}x")
    # 第一芝加哥法
    success_scenario = {'value': 200000}  # 20 亿
    failure_scenario = {'value': 10000}   # 1 亿（清算价值）
    probability_of_success = 0.3
    chicago_result = OtherValuationMethods.first_chicago_method(
        early_company,
        success_scenario,
        failure_scenario,
        probability_of_success=probability_of_success
    )
    print(f"\n{chicago_result}")
    print(f"  成功情景: {success_scenario['value']/10000:.2f}亿元")
    print(f"  失败情景: {failure_scenario['value']/10000:.2f}亿元")
    print(f"  成功概率: {probability_of_success:.0%}")
    print(f"  期望价值: {chicago_result.value/10000:.2f}亿元")
    print(f"\n{'='*60}")
    print("估值方法示例完成")
    print(f"{'='*60}")
# 辅助函数：数据验证
def validate_company_data(company: Company) -> dict:
    """
    验证公司数据的完整性
    Args:
        company: 公司对象
    Returns:
        验证结果字典
    """
    result = {
        'is_valid': True,
        'warnings': [],
        'missing_fields': []
    }
    # 检查必填字段
    if not company.name:
        result['missing_fields'].append('公司名称')
        result['is_valid'] = False
    if company.revenue <= 0:
        result['warnings'].append('营业收入为 0 或负数')
    if company.growth_rate < 0:
```

```
        result['warnings'].append('增长率为负数')
    return result
# 辅助函数：格式化估值结果
def format_valuation_result(result) -> str:
    """
    格式化估值结果用于显示
    Args:
        result: 估值结果对象
    Returns:
        格式化后的字符串
    """
    lines = [
        f"估值方法: {result.method}",
        f"估值结果: {result.value/10000:.2f}亿元",
        f"估值倍数: {result.details.get('multiple', 'N/A')}x",
    ]
    return "\n".join(lines)
# 主程序结束
```